# CSA1701 - ARTIFICIAL INTELLIGENCE

Aim: To implement the A* search algorithm to find the shortest path from a starting node to a goal node using path cost and heuristic cost.

Algorithm:

1. Initialize the starting node.
2. Calculate \(f(n)=g(n)+h(n)\), where \(g(n)\) is the path cost and \(h(n)\) is the estimated cost to the goal.
3. Select the node with the lowest \(f(n)\).
4. Explore its neighboring nodes and update their costs.
5. Continue until the goal node is reached.

Program:

*untitled (3.14.8)*

File Edit Format Run Options Window Help

```
import heapq

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 1), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 3)],
    'E': [('G', 2)],
    'F': [('G', 2)],
    'G': []
}

h = {
    'A': 5, 'B': 4, 'C': 3,
    'D': 2, 'E': 2, 'F': 1, 'G': 0
}

def astar(start, goal):
    pq = [(h[start], 0, start, [start])]
    visited = set()

    while pq:
        f, g, node, path = heapq.heappop(pq)

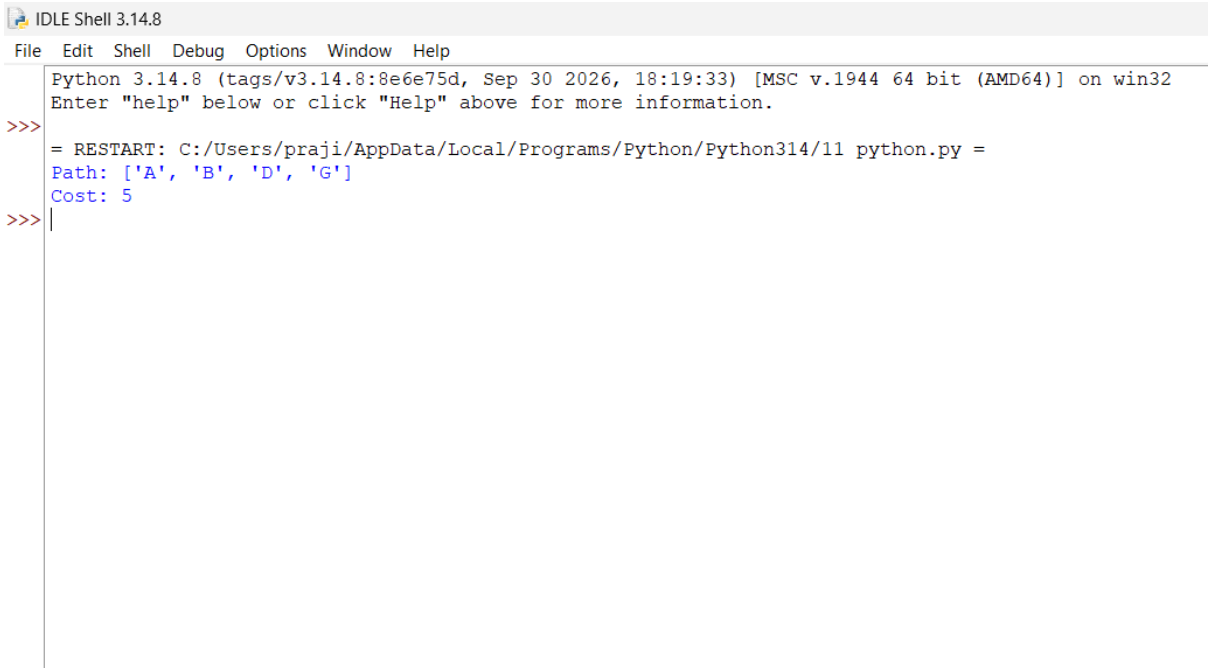
        if node == goal:
            return path, g

        if node in visited:
            continue

        visited.add(node)

        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + h[neighbor]
                heapq.heappush(
                    pq, (new_f, new_g, neighbor,
                         path + [neighbor])
                )

path, cost = astar('A', 'G')
print("Path:", path)
print("Cost:", cost)
```




Result: Thus, the A* algorithm was implemented to find a path from the starting node to the goal node.